Database Systems Project Proposal

Project Name

IS CAP : Influencer Sponsorship & Campaign Analytics Platform

Roles & Members

Role	Member
Project Lead	Mitchel Vore
Database Designer	Ethan Childs
Query Engineer	Nolan Jones

Problem Statement

Too often there are difficulties with bringing new products to market. Whether the product is sold by a big corporation with millions budgeted for advertising or by an individual with $500 in savings and genuinely innovative useful product. Our mission is to create a service which will accurately match companies big and small with reputable individuals or groups in order to facilitate the dissemination of products to those individuals which will most readily benefit.

Using systems such as excel, sheets or other manual systems tends to lead to system failures, either creating duplicates that shouldn't exist or bad information being placed due to user error. constraints such as contract per accepted application, tracking multiple deliverables per contract, monitoring engagement metrics over time, and ensuring accurate financial reporting become impossible to enforce and evaluate consistently and very difficult to correct once a mistake has been made.

This project proposes a relational database system that models the full lifecycle of influencer marketing campaigns. The system will enforce meaningful constraints, maintain data integrity, and support advanced analytical queries to evaluate campaign effectiveness, influencer performance, and return on investment.

Requirements and Target Use

Brands: Create and manage sponsorship campaigns, review influencer applications, issue contracts, track deliverables, and monitor campaign performance metrics.

Influencers: Create profiles, apply to campaigns, manage active contracts, submit deliverables, and track earnings and engagement statistics.

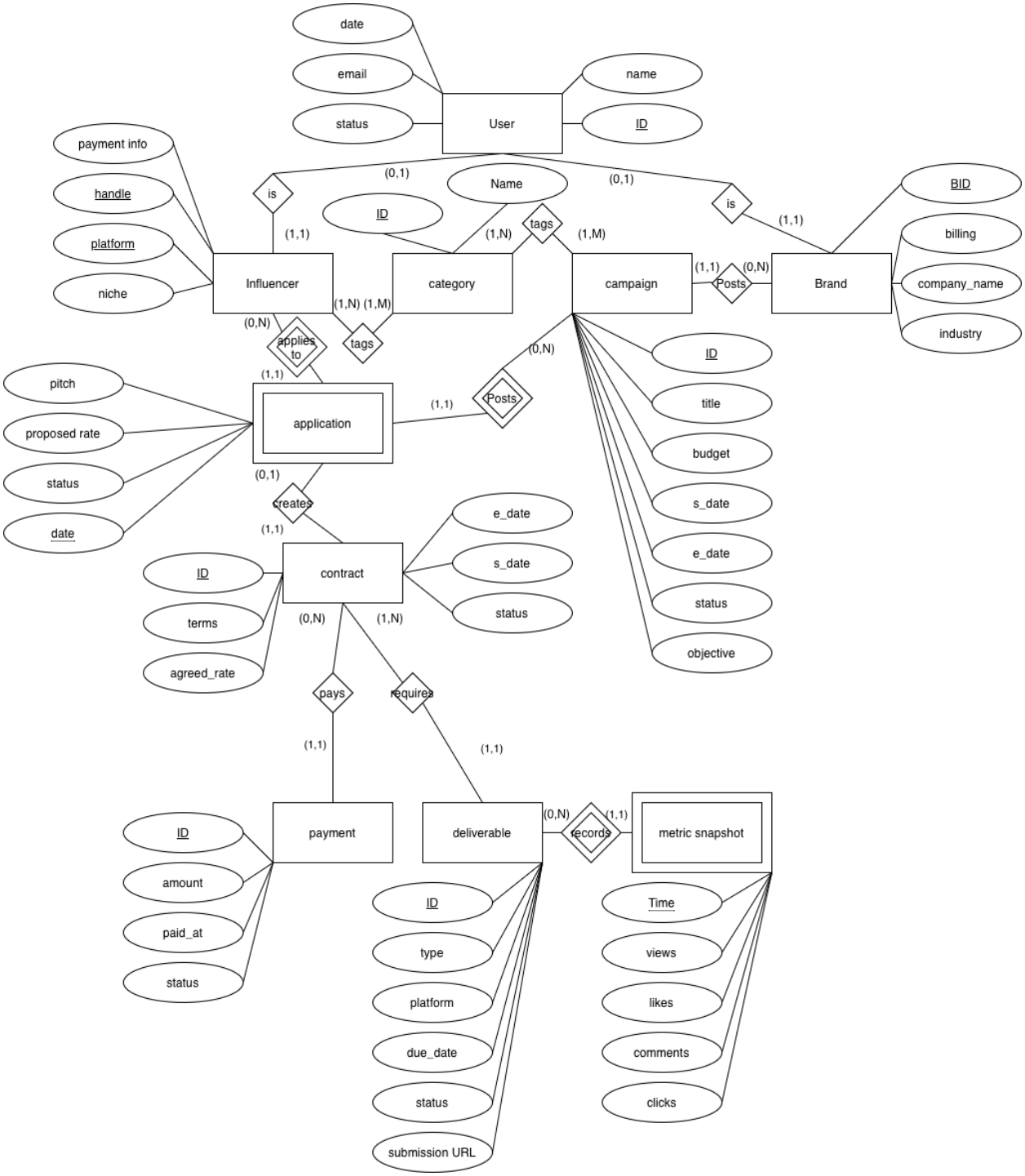
Administrators: Oversee platform activity, resolve disputes, and monitor overall system health.

Core functional requirements include:

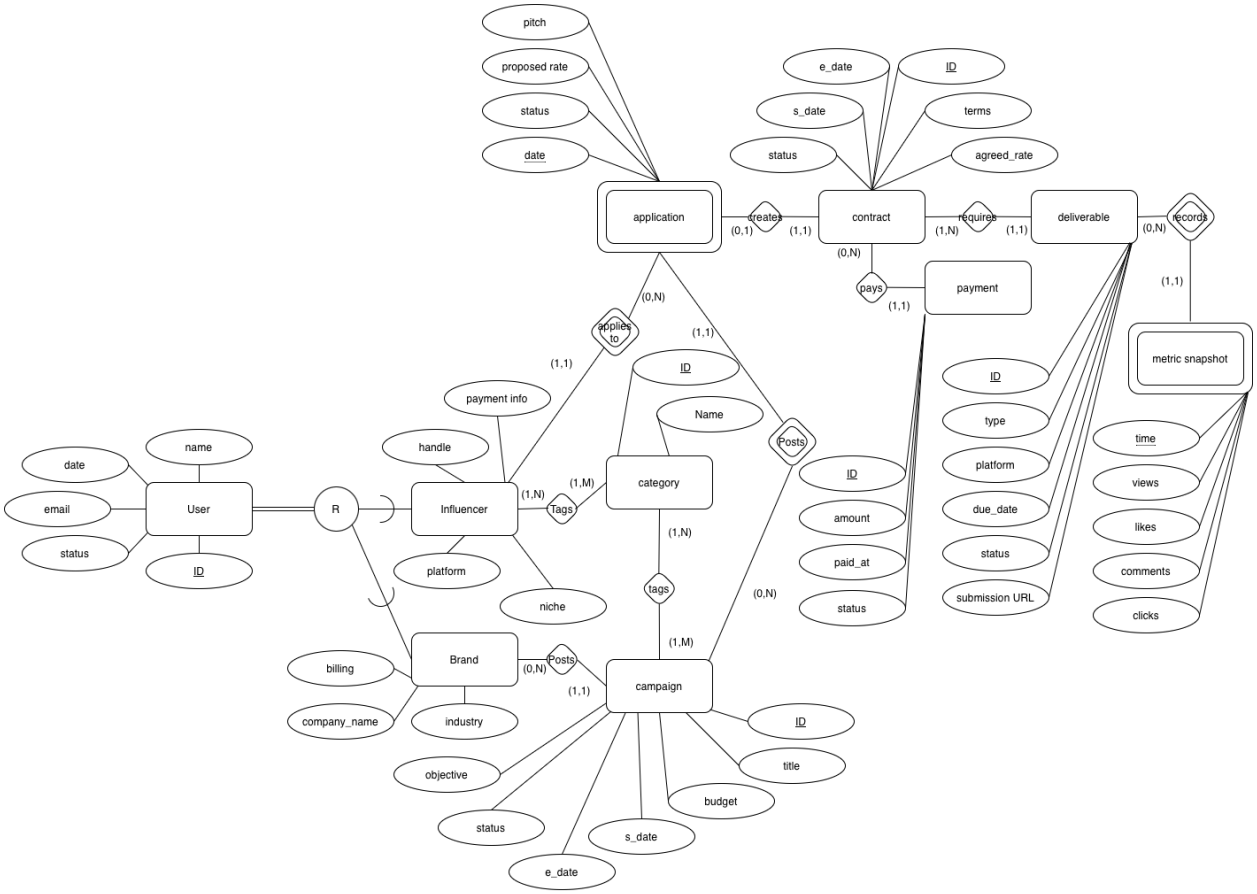
- Ability for brands to create campaigns with defined budgets, timelines, and objectives.
- Influencers can apply to multiple campaigns, and brands can review multiple applications.
- Accepted applications generate formal contracts with defined deliverables.
- Contracts may include multiple deliverables (e.g., posts, videos, stories).
- Engagement metrics (views, clicks, likes, conversions) must be recorded over time.
- Payments must be tracked and linked to contracts.
- The database must support analytical queries such as ROI calculation, top-performing influencers, and campaign performance trends.

Diagrams and Tables

ER Diagram



EER Diagram



Assumptions for cardinalities:

A user must be an influencer or brand but not both

Brands do not have to have a campaign

A campaign has exactly one brand

A campaign has 1 to many categories

An influencer has 1 to many categories

Influencers do not have to apply to applications but all applications must have an influencer

Campaigns may not have be eligible for application at a time

There is one contract per application

There are 1 to many payments for a payment but a payment may not have happened yet. Therefore it could be at 0

All contract has at least one deliverable

All deliverables will have at least one metric snapshot, however before the first snapshot has been taken it is at 0

An influencer only has one handle and one platform

Tables

Foreign key notation

Key$_{\text{Table or SuperClass/SubClass : column}}$

User

User ID	name	start_date	email	account_status	user_type

Primary Keys : User_ID
Justification: Chosen primary key

User_ID : ID for the user : hex : auto increment : non-null
name : First and Last Name : varchar : no default : non-null
start_date : date users account was created : integer/Date : current date : non-null
email : users email : varchar : no default : non-null
account_status : if account is activated, banned, locked… : int : 0 : non-null
user_type : will be used to identify as as influencer or brand exclusively : Object : no default : non-null

Influencer

User ID	handle	payment_info	niche	platform

Primary Keys : User_ID
Foreign Keys : User_ID$_{\text{User : User_ID}}$
Inherits User ID from user because it's a sub class.

handle : stored the users platform handle : varchar : no default : non-null
payment_info : bank account routing num : long long : no default : non-null
niche : influencers niche : varchar : null : allows-null
Platform : stores which platform it comes from (fb, insta, tt, yt) : varchar : no default : non-null

Brand

User ID	company_name	account_to_bill	industry

Primary Keys : User_ID
Foreign Keys : User_ID$_{\text{User : User_ID}}$
Justification: Inherits User ID from user because it's a sub class.

company_name : the companies name : varchar : no default : non-null
account_to_bill : company bank acct to bill : long long : no default : non-null
industry : primary industry the company is in : varchar : no default : non-null

Category

Category ID	name

Primary Keys : Category_ID
Justification: Chosen Primary Key

Category ID : ID for the category : hex : auto increment : non-null
name : category name : varchar : no default : non-null

Campaign

Campaign ID	title	budget	start_date	end_date	status	objective	Brand_ID

Primary Keys : Category_ID
Foreign Keys : $Brand\_ID_{Brand: User\_ID}$
Justification: Chosen Primary Key and it Connects brand to a campaign.

Campaign ID : ID for the campaign : hex : auto increment : non-null
title : campaign title : varchar : no default : non-null
budget : budget : float : no default : non-null
start_date : planned start date of campaign : int/Date : no default : non-null
end_date : planned end date of campaign : int/Date : no default : non-null
status : numerical stage representation : int : 0 : non-null
objective : ultimate goal of campaign : varchar : no default : allows-null

Application

User ID	Campaign ID	pitch	proposed_rate	status	date

Primary Keys : User_ID Campaign_ID
Foreign Keys : $User\_ID_{User : User\_ID}$, $Campaign\_ID_{Brand : Campaign\_ID}$
Justification: Inherits Primary keys from User_ID and Campaign_ID because those two entities create an application, they're also foreign because its a weak entity.

pitch : what the company wants advertised possibly step by step : varchar : no default : non-null
proposed_rate : the amount the company wishes to pay individual : float : no default : non-null
status : application status : int : 0 : non-nul
date : date of application submission : int/Date : current date : non-null

Contract

Contract ID	terms	agreed_rate	start_date	end_date	status	iUser_ID	bUser_ID

Primary Keys : Contract_ID
Foreign Keys : $iUser\_ID_{User/Influencer : User\_ID}$, $bUser\_ID_{User/Brand : User\_ID}$
Justification: Chosen Primary Key, Foreign Keys are brought from Application to identify the user and brand which are participating in the contract.

terms : terms and conditions : varchar : no-default : non-null
agreed_rate : payment rate per completed activity : float : no-default : non-null
start_date : date of first commitment : int/Date : no default : non-null
end_date : date of last commitment : int/Date : no default : non-null
status : contract terminations / has ended : 0 : non-null

Payment

Payment ID	amount	paid_at	method	status	Contract_ID

Primary Keys : Payment_ID
Foreign Keys : Contract_ID$_{\text{Contract : Contract_ID}}$
Justification: Chosen Primary Key, Foreign Key relates payment to contract

Payment ID : ID for the payment : hex : auto increment : non-null
amount : quantity paid : float : no-default : non-null
paid_at : date of payment : int\Date : no-default : non-null
method : how funds were transferred : varchar : direct-deposit : non-null
status : unpaid, processing, paid: int : 0 : non-null

Deliverable

Deliverable ID	type	platform	due_date	status	submission_URL	Contract_ID

Primary Keys : Deliverable ID
Foreign Keys : Contract_ID $_{\text{Contract : Contract\_ID}}$
Justification: Chosen Primary Key, Foreign Key relates deliverable to contract

Deliverable_ID : ID for the deliverable : hex : auto increment : non-null
type : type of deliverable image, video, url : varchar : no default : non-null
platform : which platform to complete on : varchar : no default : non-null
due_date : date to complete by/on : int/Date : no default : non-null
status : completed/missed/tbd : int : 0 ; non-null;
submission_URL : location to upload proof of completion : varchar : null : allows-null

Metric snapshots

Deliverable ID	time	views	likes	comments	clicks

Primary Keys : Deliverable_ID time
Foreign Keys : Deliverable_ID $_{\text{Deliverable : Deliverable\_ID}}$
Justification: Chosen Primary Key should be unique, Foreign Key inherits from Deliverable because its a weak entity

time : date/time of snapshot : int/Date : current time : non-null
views : number of views : int : no default : non-null
likes : number of likes : int : no default : non-null

comments : number of comments : int : no default : non-null
clicks : number of interactions with comments : int : no default : non-null

Influencer_Tags

User ID	Category ID

Primary Keys : User_ID Category_ID
Foreign Keys : User_ID $_{\text{User/Influencer : User\_ID}}$, Category_ID$_{\text{Category : Category_ID}}$
Justification: combination of User and Category is Primary Key (N to N relationship) as such they're both foreign

Campaign_Tags

Campaign ID	Category ID

Primary Keys : Campaign ID Category ID
Foreign Keys : Campaign ID$_{\text{Campaign : Campaign_ID}}$, Category ID$_{\text{Category : Category_ID}}$
Justification: combination of Campaign and Category is Primary Key (N to N relationship) as such they're both foreign

Relation Table Justifications

Foreign keys are typically added based upon the previous entity/s which caused the existence of the current one. With a rare exception of contract inheriting foreign keys from a previous layer due to application being a Weak entity. These keys have been added to illustrate the relationships between entities as described in the ER/EER diagrams. Only Many to Many relationships and sub classes spawn new tables. All other place the extra information in new columns belonging to a/the entity which was "one" (not many) in the relationship

Campaign_Tags

Campaign and Category share a many to many relationship
As many campaign_ID's can have many Categorie_ID's

Influencer_Tags

Influencers and Category share a many to many relationship

As many User_ID can add many Category_ID's, we assume a user only has one platform and one handle for simplicity

Deliverable

Deliverables are Parts of Contracts where one Contract_ID can many Deliverable_IDs

Payment

Payments are Parts of Contracts where one Contract_ID can many Payments

Metric Snapshots

Are based upon Deliverables and there could be several snapshots for a Deliverable_ID over a small span of time.

Contract

Based upon an application where a contract can have one User_ID and one Brand_ID per Contract_ID.

Payment

Are based upon Contract_ID where one Contract_ID can have many Payment_ID's

Application

Each Campaign only accepts one Application per User as such no Application should contain the same exact User_ID and Campaign_ID

Campaigns

Each Brand can have many campaigns but campaigns cannot have multiple brands only one

Influencer / Brand

Both are sub classes of User as such they receive the User_ID as a primary key

User

User is a superclass and could exclusively be of either type Influencer or of type Brand as such it contains an identifier to specify which the account is. It is assumed that an account cannot be both an Influencer and Brand account

Data

We will be generating synthetic data and then manually facilitating some data interactions and creation via interfacing.